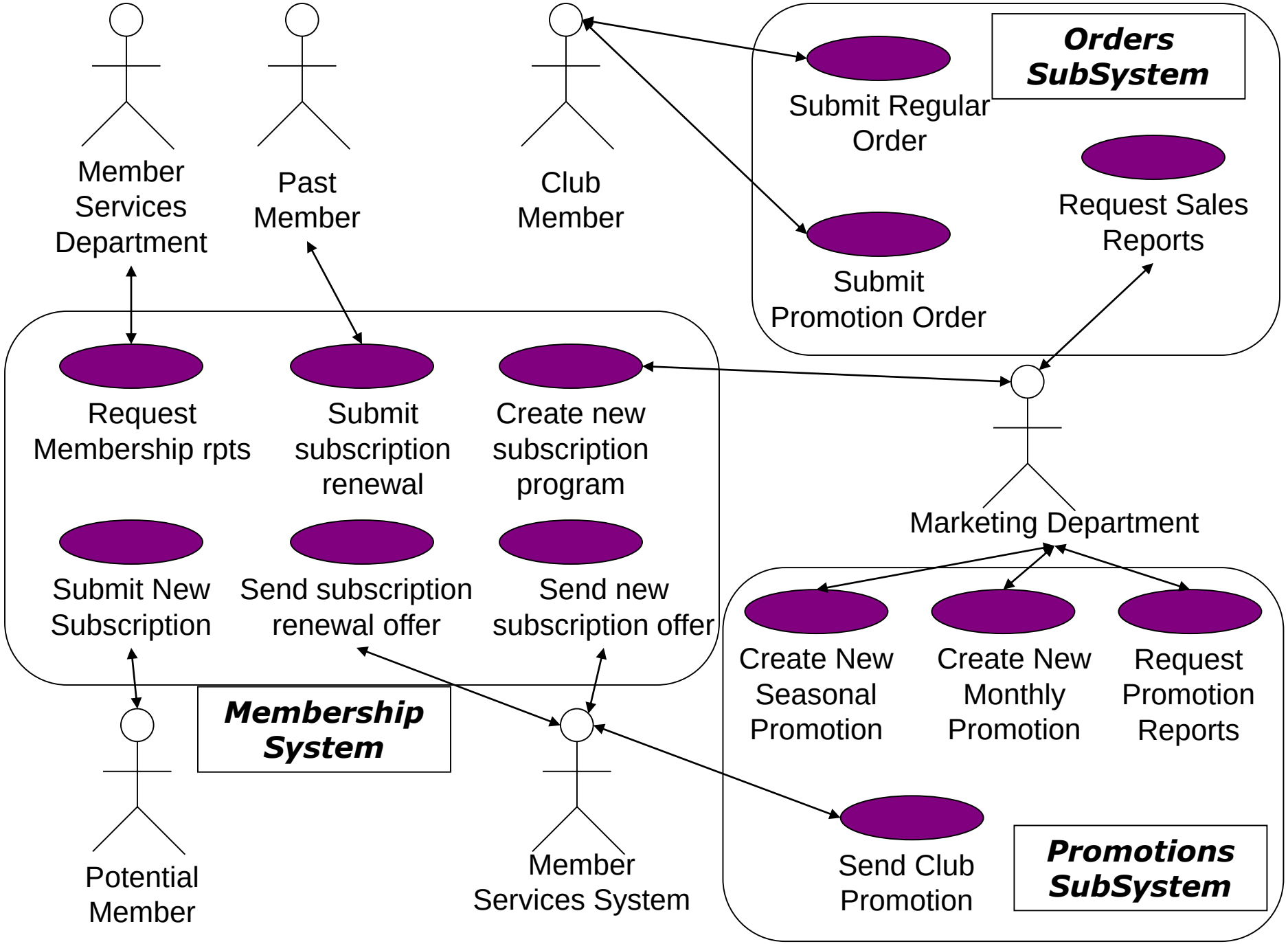
System Sequence Diagrams (SSD)

Contents

- Use Case description
- Making SSD from Use case description
- Identifying objects and operations
- SSD notation
- Examples

Use Case description

- Use case name
- Actors, brief description
- Actions taking place
- scenario – flow of events
- Some use cases have multiple scenarios to explore various contingent activities
- Preconditions –before the use case can begin
- Post-conditions: state of system and perhaps for actors, after the use case is completed


Member Services Department
Past Member
Club Member
Orders SubSystem
Submit Regular Order
Request Sales Reports
Submit Promotion Order
Request Membership rpts
Submit subscription renewal
Create new subscription program
Marketing Department
Submit New Subscription
Send subscription renewal offer
Send new subscription offer
Membership System
Create New Seasonal Promotion
Create New Monthly Promotion
Request Promotion Reports
Potential Member
Member Services System
Send Club Promotion
Promotions SubSystem

Example

USE CASE NAME	Submit Promotion Order
ACTOR	Club Member
DESCRIPTION	Describes the process when a club member submits a club promotion order to either indicate the products they are interested in ordering or declining to order during this promotion
NORMAL COURSE	1. This use is initiated when the club member submits the promotion order to be proceeded
	2. The club member's personal information such as address is validated against what is currently recorded in member services
	3. The promotion order is verified to see if product is being ordered
	4. The club member's credit status is checked with Accounts Receivable to make sure no payments are outstanding
	5. For each product being ordered, validate the product number
	6. For each product being ordered, check the availability in inventory and record the ordered information which includes "quantity being ordered" and give each ordered product a status of "open"
	7. Create a Picking Ticket for the promotion order containing all ordered products which have a status "open"
	8. Route the picking ticket to the warehouse

Creating SSD

How to construct an SSD from a use case:

- Actors
- Identify Nouns: *Objects*
- Identify verbs: *Interaction between Objects*

Example

- The *SafeHome* security function enables the homeowner to configure the security system when it is installed, monitors all sensors connected to the security system, and interacts with the homeowner through the Internet, a PC, or a control panel.
- During installation, the *SafeHome* PC is used to program and configure the system. Each sensor is assigned a number and type, a master password is programmed for arming and disarming the system, and telephone number(s) are input for dialing when a sensor event occurs.
- When a sensor event is recognized, the software invokes an audible alarm attached to the system. After a delay time that is specified by the homeowner during system configuration activities, the software *dials* a telephone number of a monitoring service, provides information about the location, reporting the nature of the event that has been detected. The telephone number will be redialed every 20 seconds until a telephone connection is obtained.
- The homeowner receives security information via a control panel, the PC, or a browser, collectively called an interface. The interface displays prompting messages and system status information on the control panel, the PC, or the browser window. Homeowner interaction takes the following form…

Identifying objects

- The ***SafeHome* security function** enables the **homeowner** to configure the **security system** when it is installed, monitors all **sensors** connected to the security system, and interacts with the homeowner through the **Internet**, a **PC**, or a **control panel**.
- During **installation**, the *SafeHome* PC is used to program and configure the system. Each sensor is assigned a **number** and **type**, a **master password** is programmed for arming and disarming the system, and **telephone number(s)** are input for dialing when a **sensor event** occurs.
- When a sensor event is recognized, the software invokes an audible **alarm** attached to the system. After a **delay time** that is specified by the homeowner during system configuration activities, the software *dials* a telephone number of a **monitoring service**, provides information about the **location**, reporting the nature of the event that has been detected. The telephone number will be redialed every 20 seconds until a **telephone connection** is obtained.
- The homeowner receives **security information** via a control panel, the PC, or a browser, collectively called an **interface**. The interface displays prompting **messages** and system **status information** on the control panel, the PC, or the browser window. Homeowner interaction takes the following form…

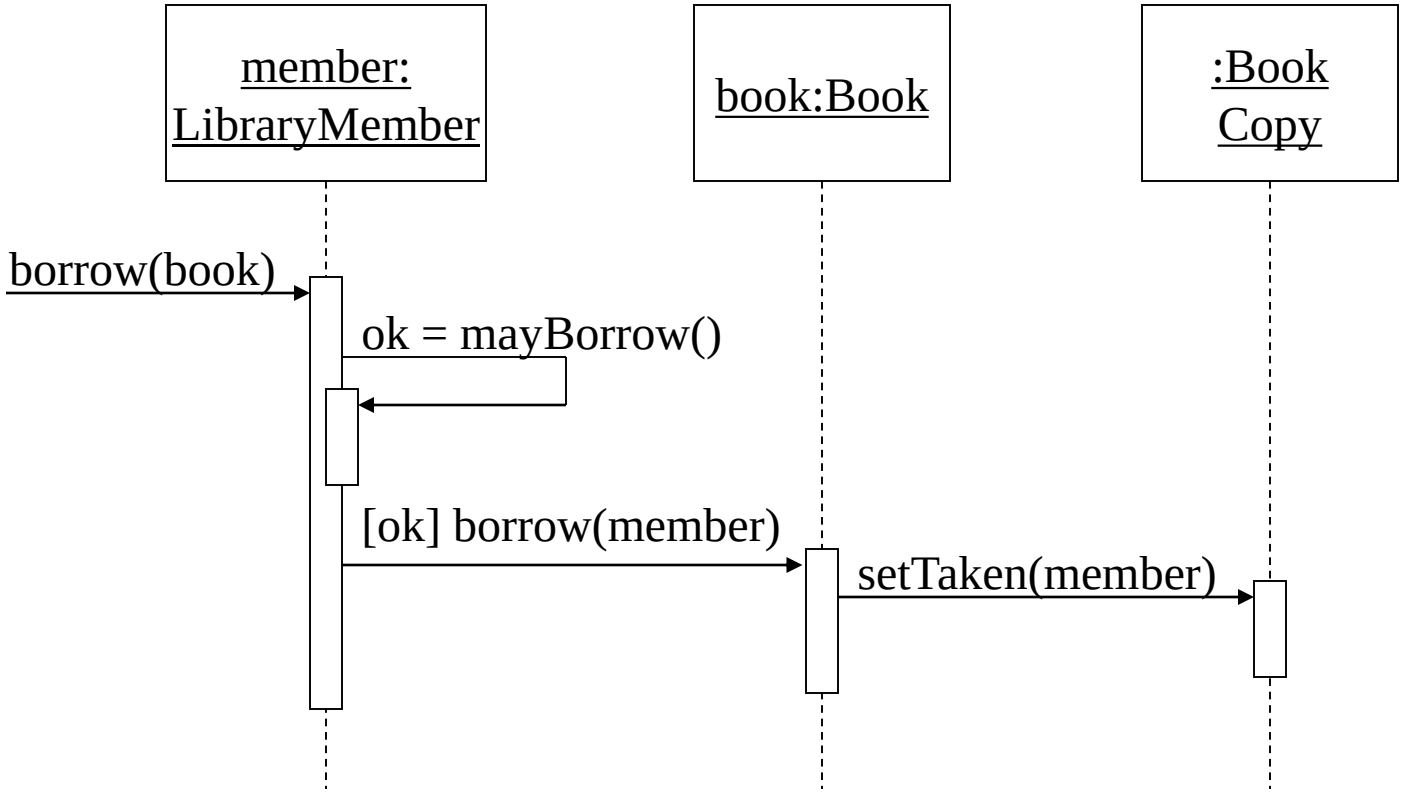
Identifying operations

- The *SafeHome* security function enables the homeowner to configure the security system when it is installed, monitors all sensors connected to the security system, and interacts with the homeowner through the Internet, a PC, or a control panel.
- During installation, the *SafeHome* PC is used to program and configure the system. Each sensor is assigned a number and type, a master password is programmed for arming and disarming the system, and telephone number(s) are input for dialing when a sensor event occurs.
- When a sensor event is recognized, the software invokes an audible alarm attached to the system. After a delay time that is specified by the homeowner during system configuration activities, the software *dials* a telephone number of a monitoring service, provides information about the location, reporting the nature of the event that has been detected. The telephone number will be redialed every 20 seconds until a telephone connection is obtained.
- The homeowner receives security information via a control panel, the PC, or a browser, collectively called an interface. The interface displays prompting messages and system status information on the control panel, the PC, or the browser window. Homeowner interaction takes the following form…

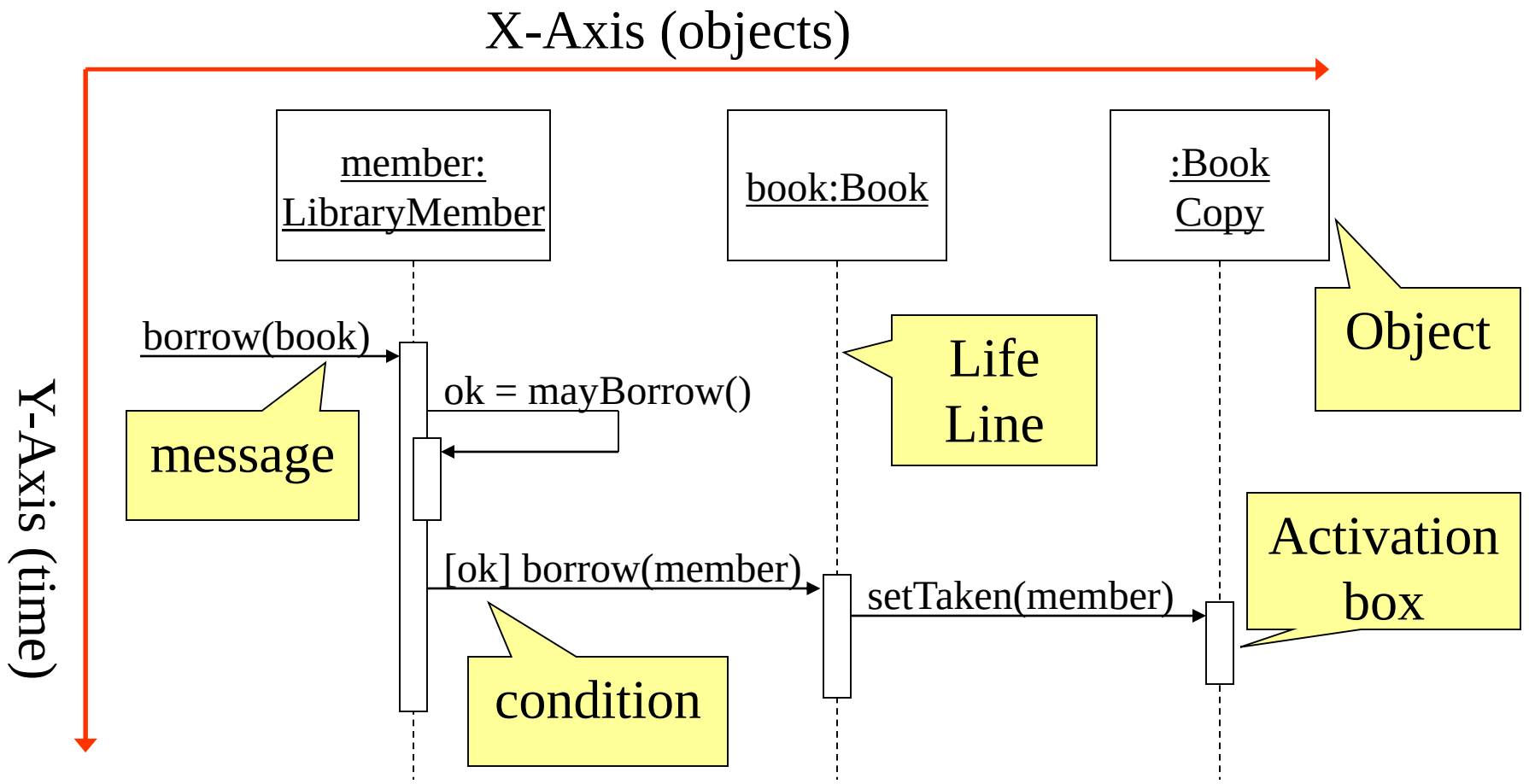
A First Look at Sequence Diagrams

- Illustrates how objects interacts with each other.
- Emphasizes time ordering of messages.
- Can model simple sequential flow, branching, iteration, recursion and concurrency.

A Sequence Diagram

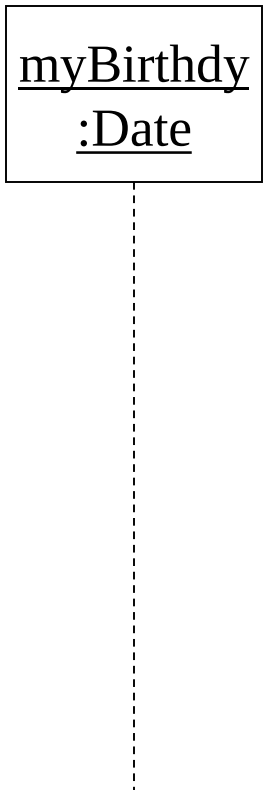


A Sequence Diagram



Object

- Object naming:
  - syntax: *[instanceName][:className]*
  - Name classes consistently with your class diagram (same classes).
  - Include instance names when objects are referred to in messages or when several objects of the same type exist in the diagram.
- The *Life-Line* represents the object's life during the interaction



Messages

- An interaction between two objects is performed as a message sent from one object to another (simple operation call, Signaling, RPC)
- If object obj_1 sends a message to another object obj_2 some link must exist between those two objects (dependency, same objects)

Messages (Cont.)

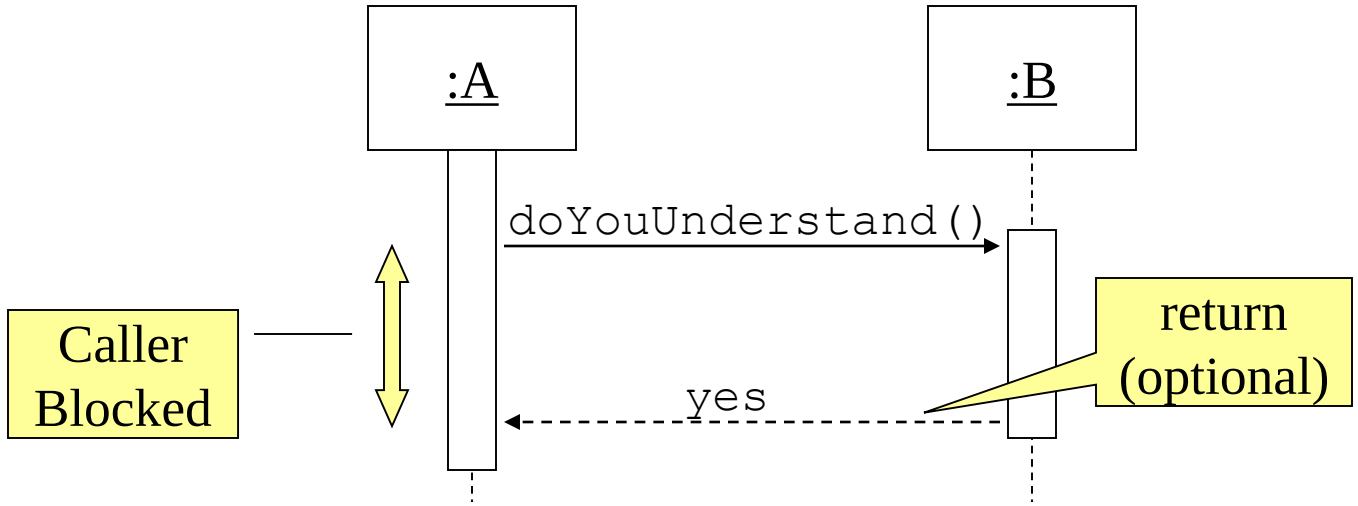
- A message is represented by an arrow between the life lines of two objects.
  - Self calls are also allowed
  - The time required by the receiver object to process the message is denoted by an *activation-box.*
- A message is labeled at minimum with the message name.
  - Arguments and control information (conditions, iteration) may be included.

Return Values

- Optionally indicated using a dashed arrow with a label indicating the return value.
  - Don’t model a return value when it is obvious what is being returned, e.g. getTotal()
  - Model a return value only when you need to refer to it elsewhere, e.g. as a parameter passed in another message.
  - Prefer modeling return values as part of a method invocation, e.g. `ok = isValid()`

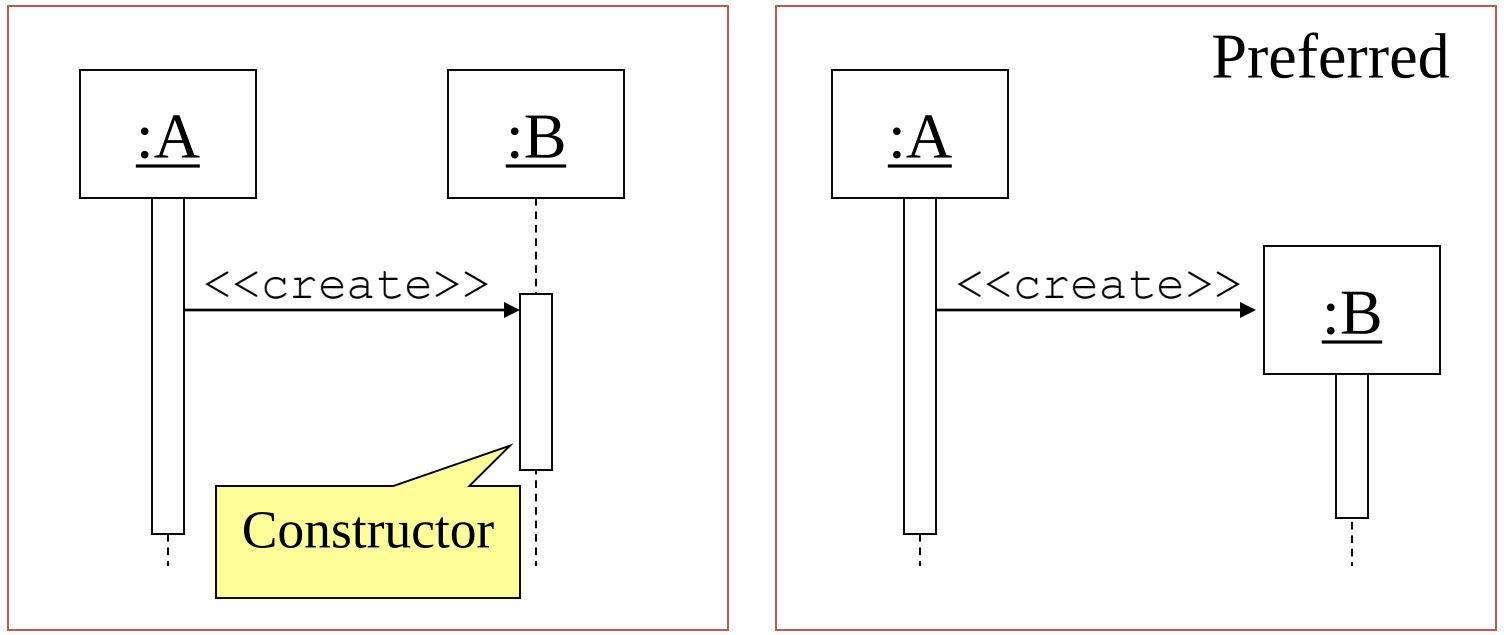
Synchronous Messages

- Nested flow of control, typically implemented as an operation call.
  - The routine that handles the message is completed before the caller resumes execution.



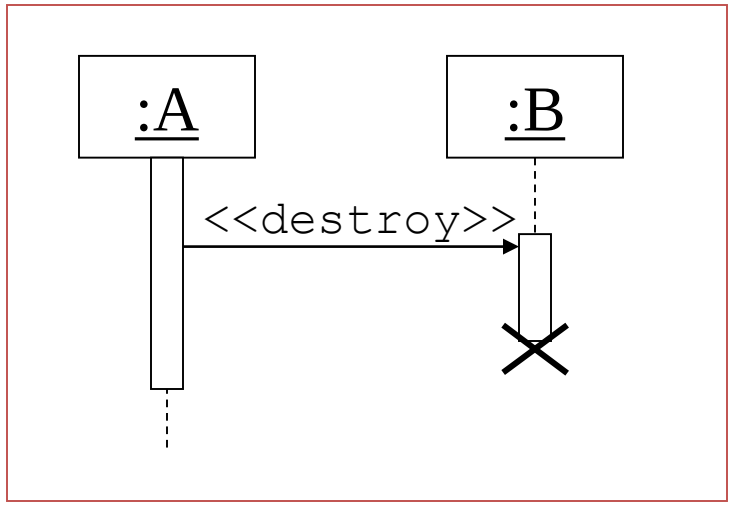
Object Creation

- An object may create another object via a `<<create>>` message.



Object Destruction

- An object may destroy another object via a `<<destroy>>` message.
  - An object may destroy itself.
  - Avoid modeling object destruction unless memory management is critical.



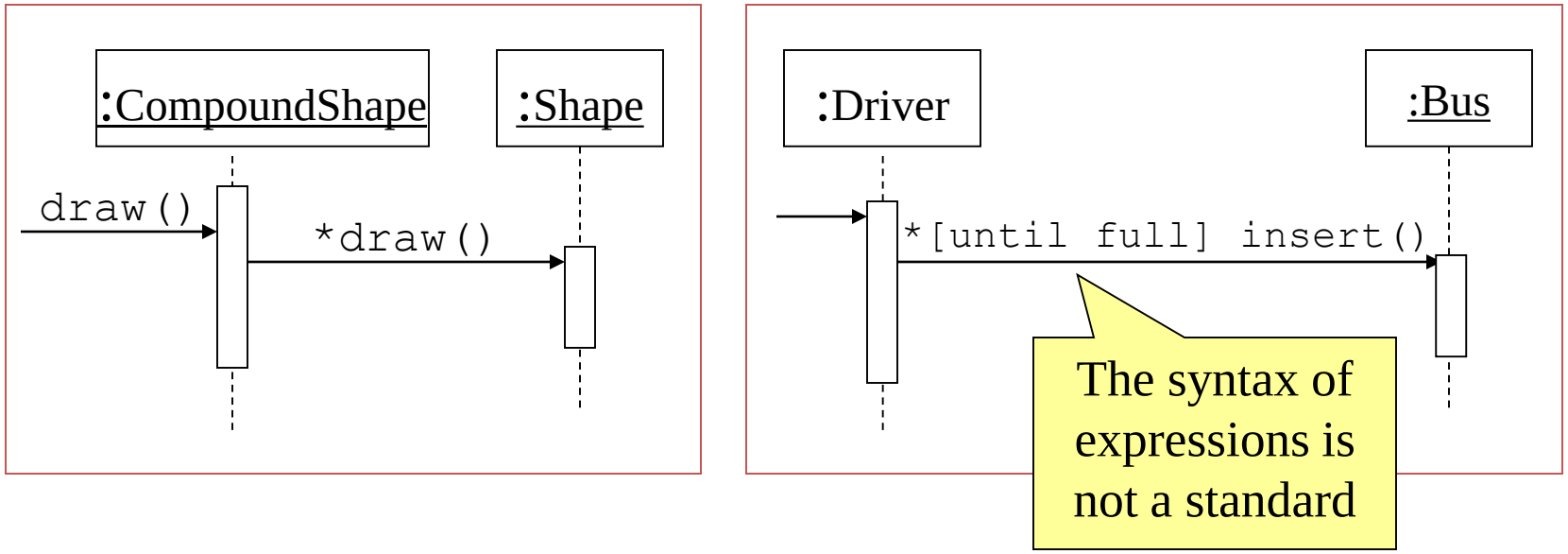
Control information

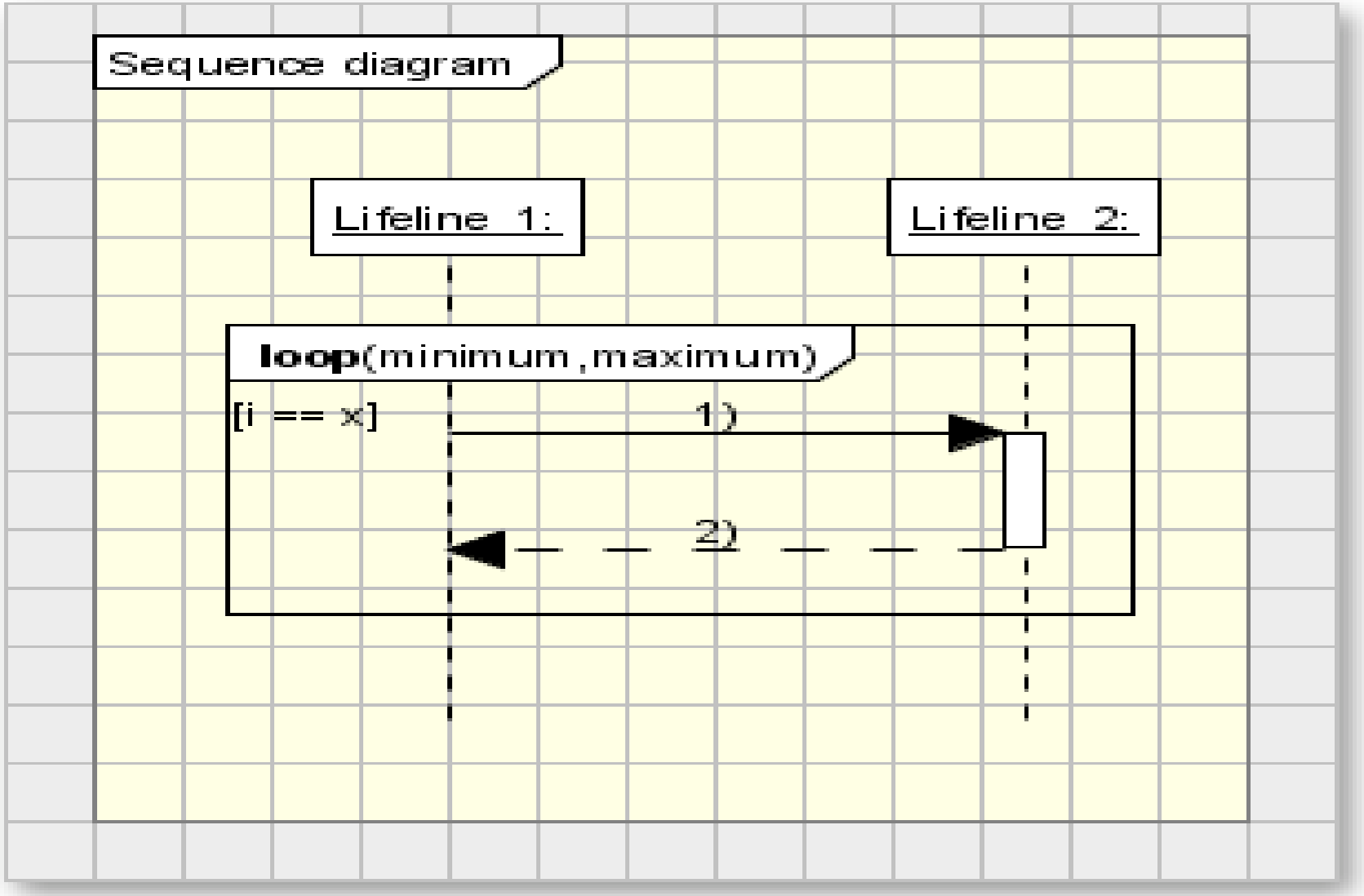
- Condition
  - syntax: ‘[‘ expression ’]’ message-label
  - The message is sent only if the condition is true
  - example: 

- Iteration
  - syntax: * [‘[‘ expression ‘]’] message-label
  - The message is sent many times to possibly multiple receiver objects.

Control Information (Cont.)

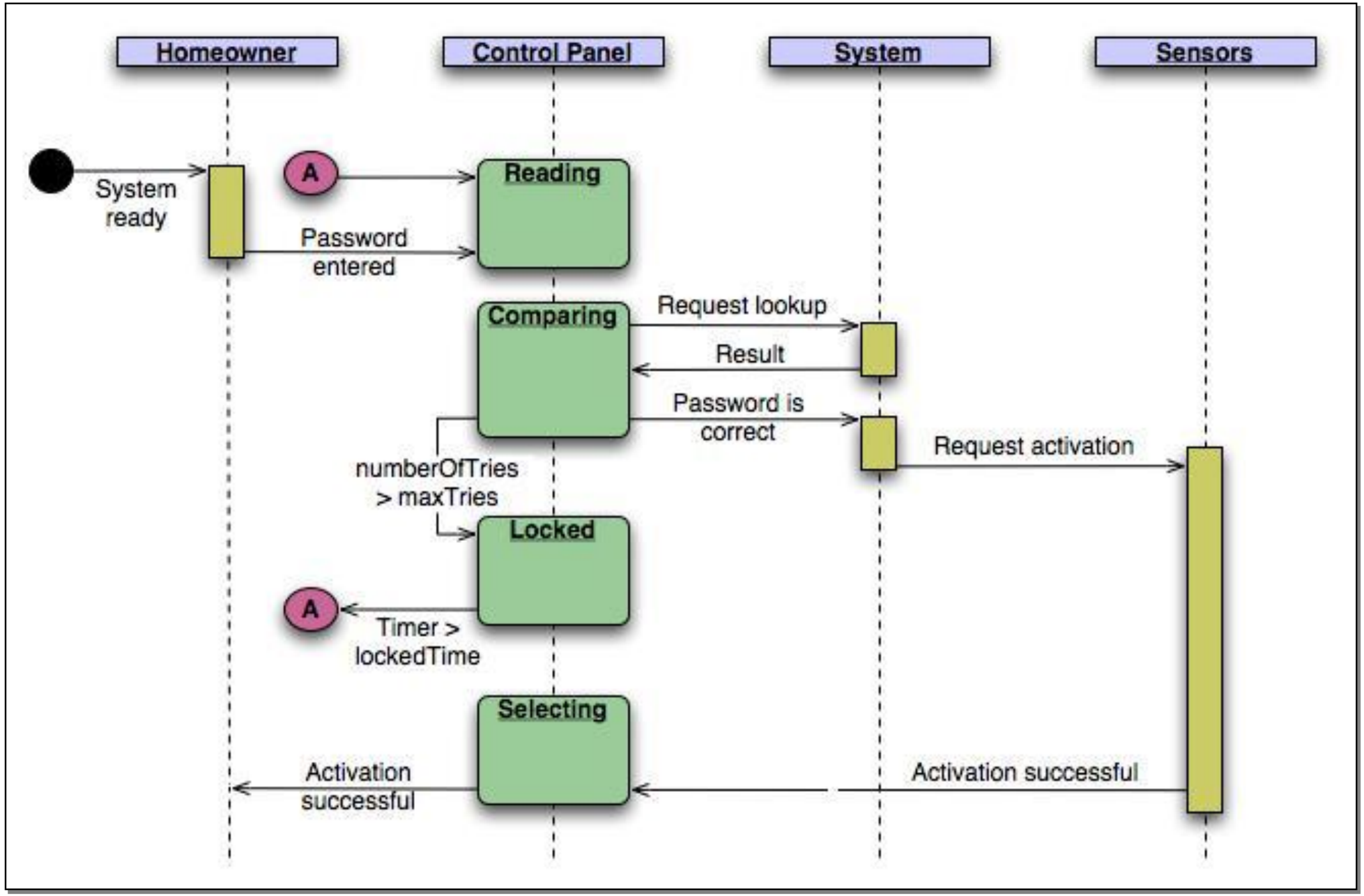
- Iteration examples:



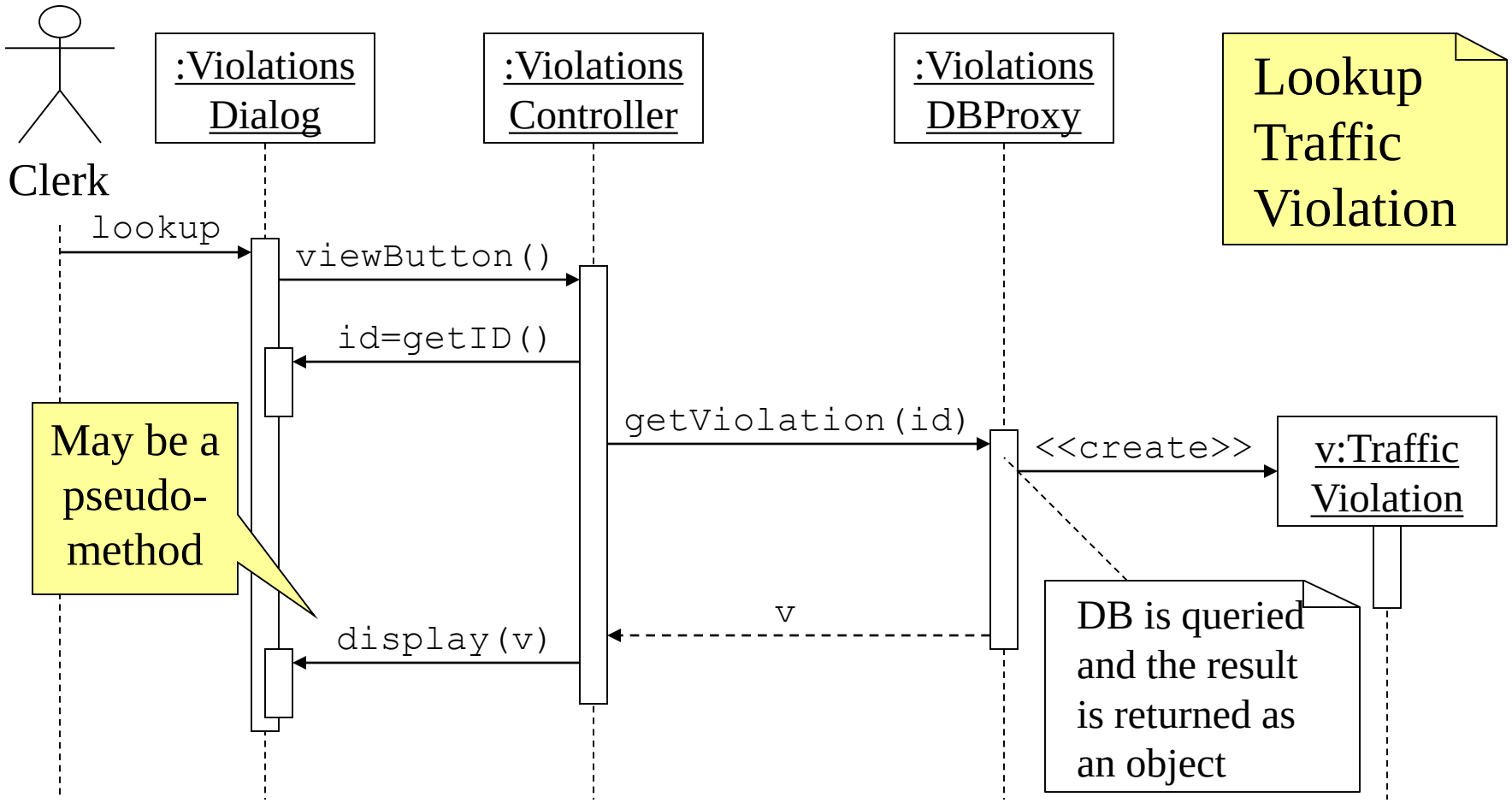

Sequence diagram
Lifeline 1:
Lifeline 2:
loop(minimum,maximum)
[i == x]
1)
2)

Control Information (Cont.)

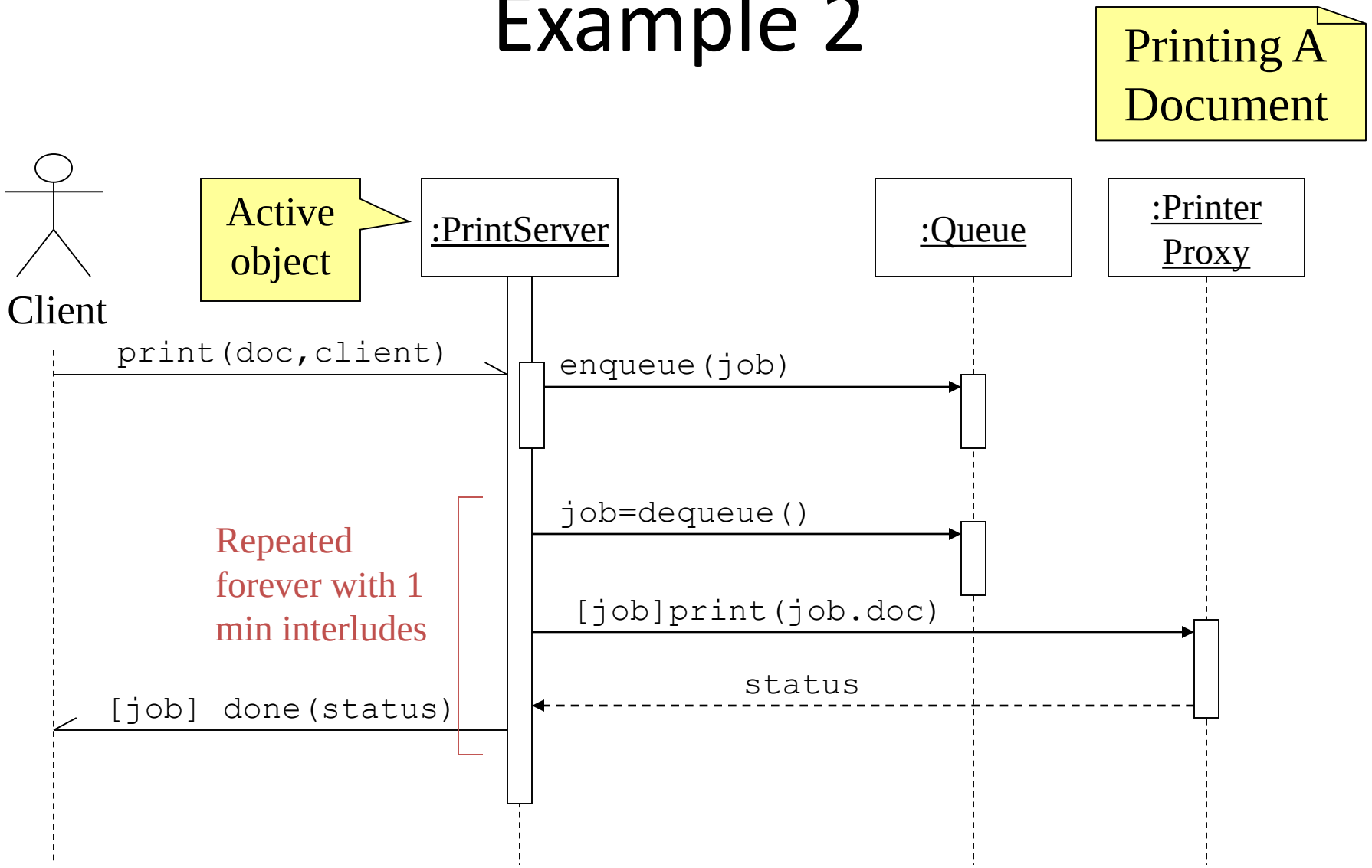
- The control mechanisms of sequence diagrams suffice only for modeling simple alternatives.
  - Consider drawing several diagrams for modeling complex scenarios.
  - Don’t use sequence diagrams for detailed modeling of algorithms (this is better done using *activity diagrams, pseudo-code* or *state-charts*).

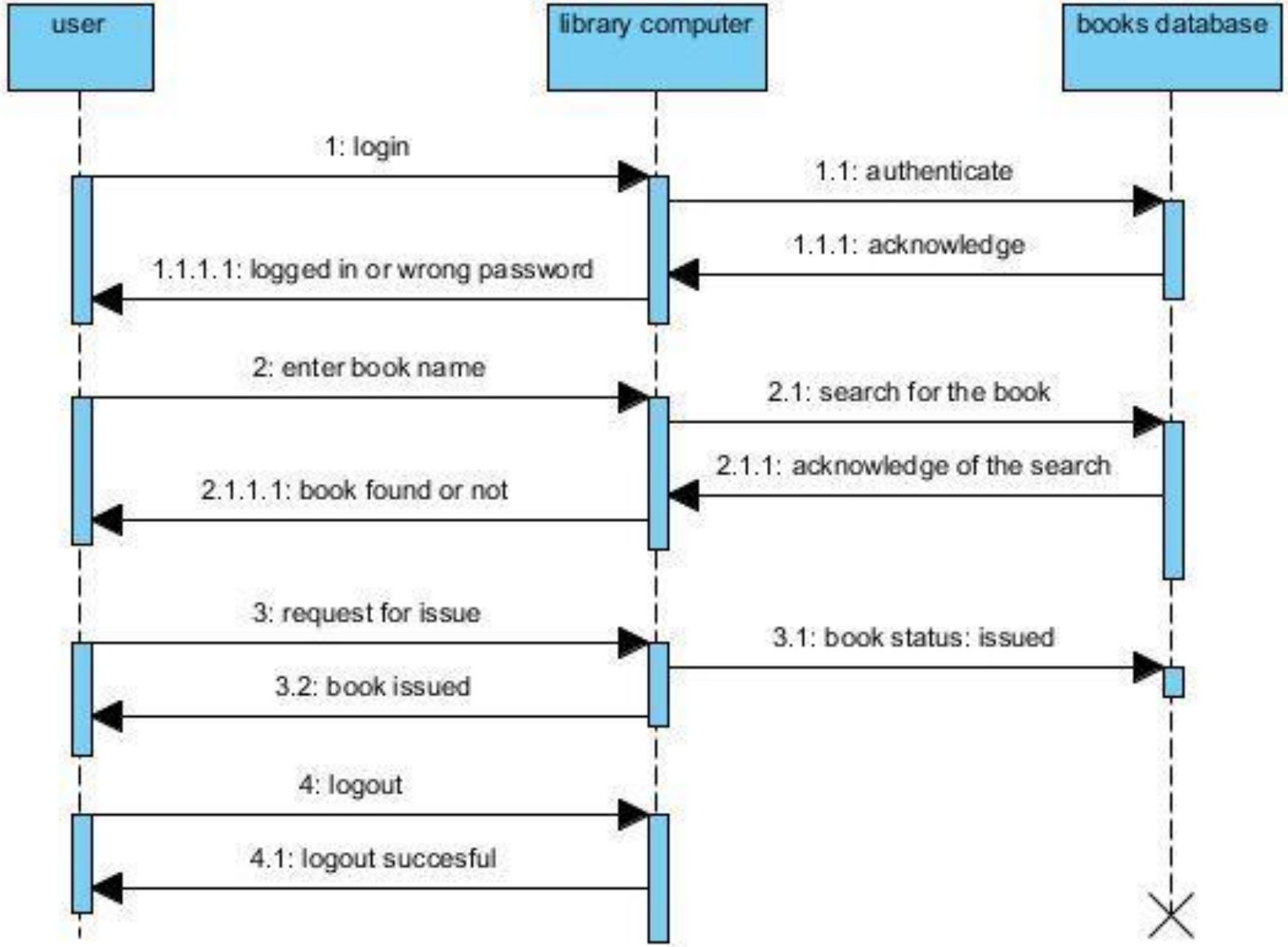

Homeowner
Control Panel
System
Sensors
System ready
A
Reading
Password entered
Comparing
Request lookup
Result
Password is correct
Request activation
numberOfTries > maxTries
Locked
A
Timer > lockedTime
Selecting
Activation successful
Activation successful

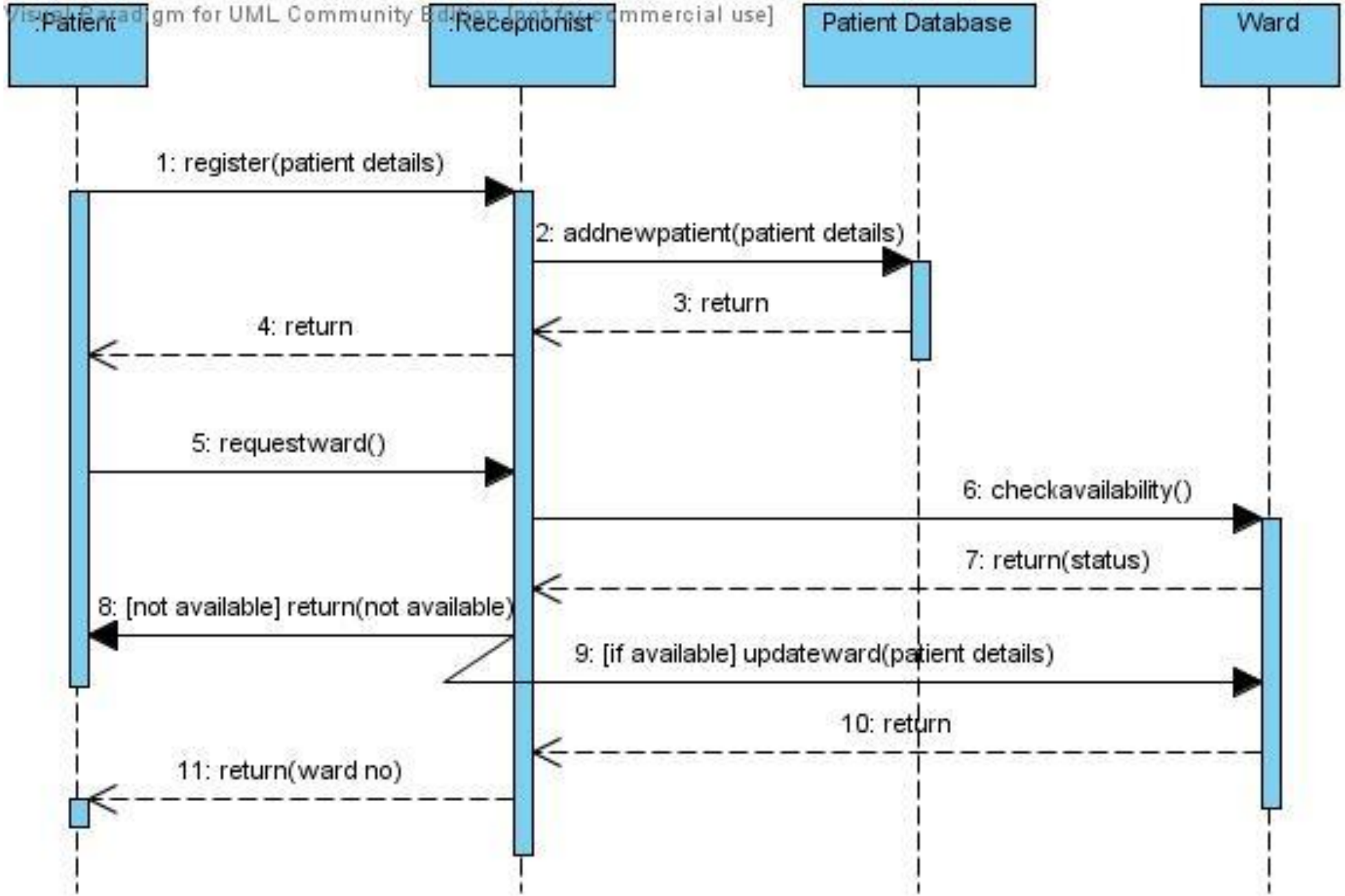
Example 1

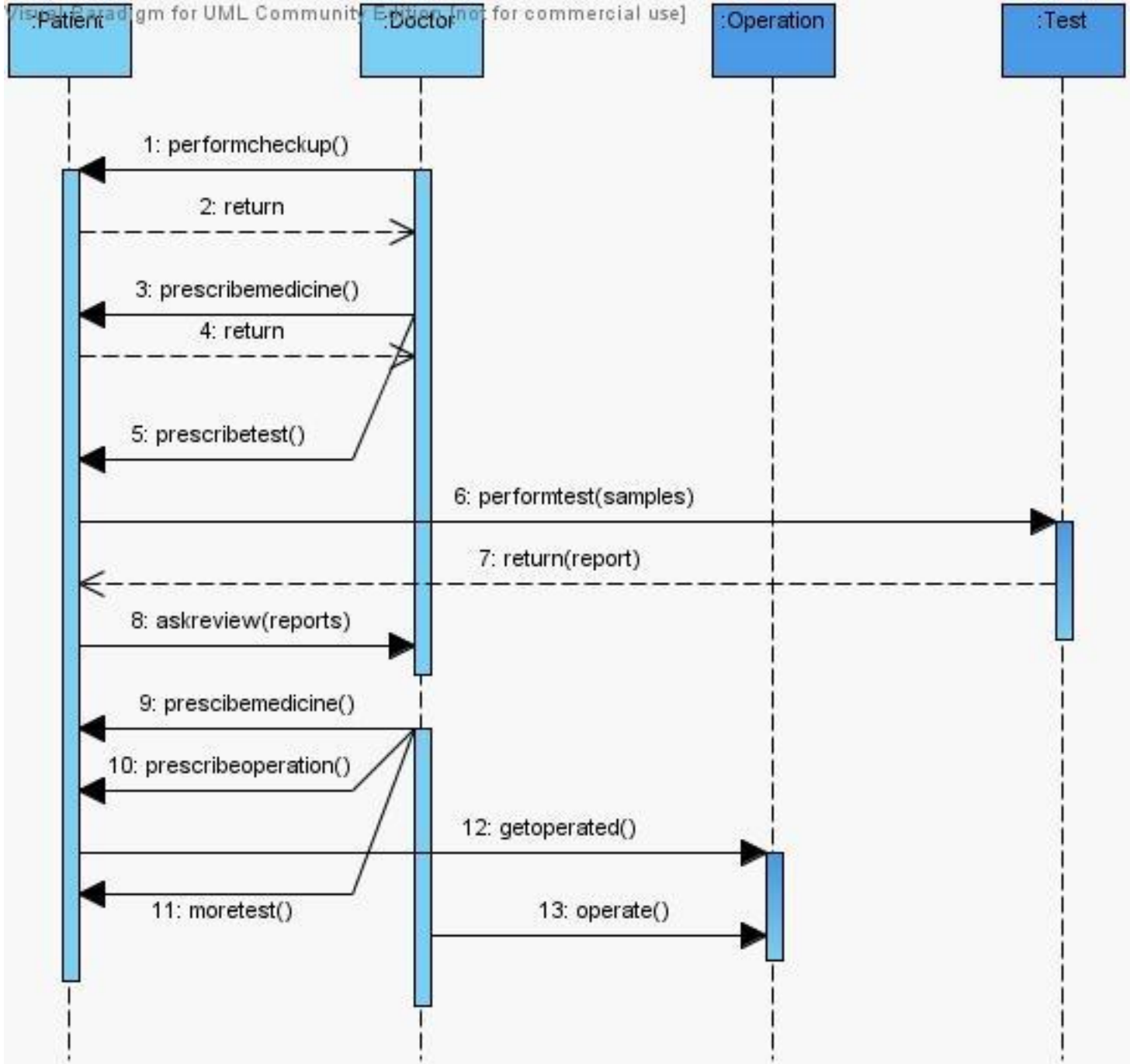

Clerk
:Violations Dialog
:Violations Controller
:Violations DBProxy
Lookup Traffic Violation
lookup
viewButton()
id=getID()
getViolation(id)
<<create>>
v:Traffic Violation
May be a pseudo-method
v
display(v)
DB is queried and the result is returned as an object

Example 2


Printing A Document
Client
Active object
:PrintServer
:Queue
:Printer Proxy
print(doc,client)
enqueue(job)
job=dequeue()
Repeated forever with 1 min interludes
[job]print(job.doc)
status
[job] done(status)


user
library computer
books database
1: login
1.1: authenticate
1.1.1: acknowledge
1.1.1.1: logged in or wrong password
2: enter book name
2.1: search for the book
2.1.1: acknowledge of the search
2.1.1.1: book found or not
3: request for issue
3.1: book status: issued
3.2: book issued
4: logout
4.1: logout succesful





Summary

- Use Case description
- Making SSD from Use case description
- Identifying objects and operations
- SSD notation
- Examples